

Guide Projet Spring - Blog Henock

Guide complet — Projet Spring Boot

Blog Henock (Tumonakiese Henock)

Ce document explique à quoi sert le projet, comment il a été conçu, et ce qu'il faut retenir sur Spring Boot pour le comprendre et le présenter.

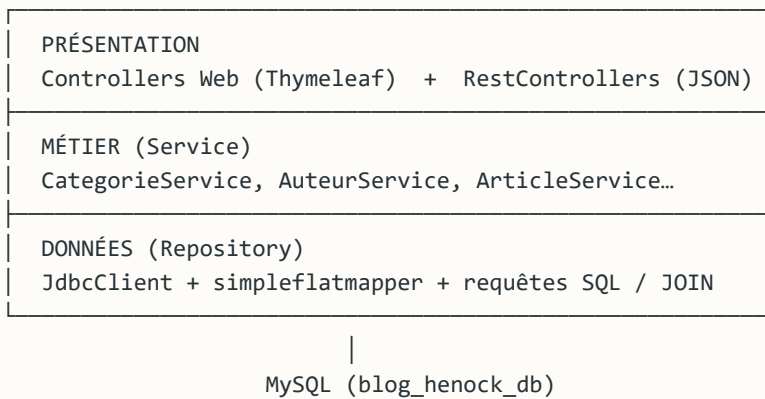
1. De quoi parle ce projet ?

Application web de **gestion de blog** : catégories, auteurs, articles et commentaires.

Entité	Rôle métier	Exemple
Catégorie	Thème du blog	Technologie, Voyage, Cuisine
Article	Publication du blog	« Spring Boot en 2026 : par où commencer ? », 1240 vues
Auteur	Rédacteur	Tumonakiese Henock, Lubumbashi, 3 ans d'expérience
Commentaire	Retour d'un lecteur	« Article très clair, merci pour les exemples Spring ! »

L'application propose **deux façons** d'utiliser les données : - **Interface web** (navigateur) : formulaires, tableaux, boutons - **API REST** (JSON) : pour tester ou consommer les données depuis un autre programme

2. Architecture en 3 couches (Spring)



Pourquoi 3 couches ?

Couche	Responsabilité	Ne doit pas...
Controller	Recevoir la requête HTTP, renvoyer une page ou du JSON	Contenir de la logique SQL
Service	Règles métier, validation, transactions	Connaître HTML ou HTTP
Repository	Lire/écrire en base de données	Valider les formulaires

Exemple concret : quand tu publies un commentaire : 1. `CommentaireController` reçoit le formulaire → `CommentaireDto` 2. `CommentaireService` valide et appelle le repository 3. `CommentaireRepoImpl` exécute `INSERT INTO commentaires ...` 4. Redirection vers la liste avec message de succès

3. Comment on a procédé pour la conception

Le projet a été construit **étape par étape**, du bas vers le haut :

Étape 0 — Préparation

- Choix du **domaine** : blog (catégories, auteurs, articles, commentaires)
- Création des scripts SQL dans le dossier `sql/` à la racine
- Configuration Maven (`pom.xml`) : Spring Web, JDBC, Thymeleaf, validation, simpleflatmapper
- Configuration `application.properties` : port **8081**, profil `dev`, MySQL

Étape 1 — Base de données

- 4 tables avec **clés étrangères** et **index**
- Relation : un **auteur** appartient à une **catégorie** (JOIN)
- Relation : un **commentaire** lie un **auteur** et un **article** (avec champ `texte`)

Étape 2 — Couche Repository

- Interface `XxxRepoCustom` + implémentation `XxxRepoImpl`

- Accès données avec `JdbcClient` (Spring 6+)
- **simpleflatmapper** pour mapper les JOIN en objets Java imbriqués
Ex. : `auteur.categorie.libelle` rempli automatiquement depuis la requête SQL

Étape 3 — Services et DTOs

- **DTO** (`AuteurDto`, etc.) : validation avec `@NotBlank`, `@Email`, `@Positive...`
- **Modèles** (`Auteur`, etc.) : alignés sur les tables
- **Mapper** : conversion DTO ↔ modèle
- **Service** : `@Transactional` sur create / update / delete

Étape 4 — API REST

- `@RestController` sur `/api/categories`, `/api/auteurs`, `/api/articles`, `/api/commentaires`
- Méthodes HTTP : GET, POST, PUT, DELETE
- Réponses JSON + codes HTTP (200, 201, 404...)

Étape 5 — Interface Web

- **Thymeleaf** : pages HTML générées côté serveur
- Un module = liste (`*-view.html`) + formulaire (`*-form.html`)
- Design blog : violet + turquoise + orange, polices Lora / Nunito Sans

Étape 6 — Finitions

- Messages flash (succès / erreur)
 - Recherche auteurs par mot-clé (`?keyword=Henock`)
 - Tests d'intégration (`ApplicationIntegrationTest`)
 - Sécurité ouverte en dev (`SecurityConfig` → `permitAll`)
-

4. Structure des dossiers

GUIDE-PROJET.md	← ce document (racine)
GUIDE-PROJET.pdf	← version PDF
sql/	← scripts base de données (indispensables)
├─ schema.sql	
├─ data.sql	
├─ reset-data.sql	
└─ fix-encoding.sql	
src/main/java/edu/upc/	
├─ BlogHenockApplication.java	← point d'entrée Spring Boot
├─ config/	← AppConfig, SecurityConfig, NavModelAdvice
├─ controllers/	← pages web (@Controller)
├─ controllers/api/	← API REST (@RestController)
├─ models/	← entités (Auteur, Article...)
├─ models/dtos/	← objets de transfert (validation)
├─ repositories/	← accès MySQL (JdbcClient)
├─ services/	← logique métier
└─ utils/	← Mapper, JdbcSfmHelper, exceptions
src/main/resources/	
├─ application.properties	
├─ application-dev.properties	
├─ Bundle.properties	← textes français de l'interface
├─ static/css/app.css	← design blog
└─ templates/	← pages HTML Thymeleaf

5. La base de données expliquée

Schéma relationnel

```
categories (1) ———< (N) auteurs
                        |
                        | (N)
                        v
articles (1) <———— (N) commentaires (N) >———— (1) auteurs
```

Fichiers SQL dans `sql/`

Fichier	Utilité
schema.sql	Crée la base <code>blog_henock_db</code> et les 4 tables
data.sql	Insère les données de démonstration
reset-data.sql	Vide tout et recharge les données propres
fix-encoding.sql	Corrige les accents si MySQL affiche des <code>?</code>

Exemple de JOIN (dans `AuteurRepoImpl`)

```
SELECT a.id, a.noms, a.ville, a.email, a.experience,
       cat.id categorie_id, cat.libelle categorie_libelle
FROM auteurs a
INNER JOIN categories cat ON a.categorie_pk = cat.id
```

`simpleflatmapper` lit `categorie_id` et `categorie_libelle` et remplit `auteur.categorie` dans l'objet Java.

Exemple de JOIN triple (commentaires)

```
SELECT v.id, v.texte,
       c.noms auteur_noms,
       b.titre article_titre, b.vues article_vues
FROM commentaires v
INNER JOIN auteurs c ON v.auteur_pk = c.id
INNER JOIN articles b ON v.article_pk = b.id
```

6. Ce qu'il faut retenir sur Spring Boot

Concepts clés

Concept	Dans ce projet
<code>@SpringBootApplication</code>	Lance le serveur embarqué Tomcat
<code>@Controller</code>	Retourne une vue HTML (<code>"articles-view"</code>)
<code>@RestController</code>	Retourne du JSON
<code>@Service</code>	Logique métier
<code>@Repository</code>	Accès base de données
<code>@Autowired</code>	Injection de dépendances
<code>@Valid + DTO</code>	Validation des formulaires / JSON
<code>@Transactional</code>	Toute la méthode réussit ou rien n'est enregistré
Thymeleaf <code>th:text="#{...}"</code>	Textes depuis <code>Bundle.properties</code>
<code>JdbcTemplate</code>	API moderne pour exécuter du SQL
Profil <code>dev</code>	Configuration locale dans <code>application-dev.properties</code>

Flux d'une requête web

```
Navigateur → GET /articles → ArticleController.list()
           → ArticleService.get()
           → ArticleRepoImpl.get()
           → MySQL
           ← Liste d'Article
           ← Template articles-view.html
           ← HTML affiché
```

Flux d'une requête API

```
Navigateur → GET /api/auteurs → AuteurRestController.get()
           → AuteurService
           → AuteurRepoImpl.getWithCategorie()
           ← JSON [{ "id": 1, "noms": "...", "categorie": { ... } }]
```

7. API REST — récapitulatif

Méthode	URL	Action
GET	/api/categories	Liste des catégories
POST	/api/categories	Créer une catégorie
PUT	/api/categories/{id}	Modifier
DELETE	/api/categories/{id}	Supprimer
GET	/api/auteurs	Liste (avec catégorie en JOIN)
GET	/api/auteurs? keyword=Henock	Recherche par nom
GET	/api/articles	Liste des articles
GET	/api/commentaires	Liste des commentaires (auteur + article)
POST	/api/commentaires	Publier un commentaire

Port par défaut : <http://localhost:8081>

8. `sql/` et Postman — sont-ils obligatoires ?

Le dossier `sql/` (à la racine) — quasi indispensable

Note : dans ce projet, les scripts SQL sont dans `sql/` à la racine, **pas** dans `docs/sql/`.
C'est le même rôle : permettre à quelqu'un d'autre de recréer la base facilement.

Question	Réponse
Obligatoire pour compiler ?	Non — Maven compile sans MySQL
Obligatoire pour faire tourner l'app ?	Oui — sans tables, l'app plante
Obligatoire à l'examen / soutenance ?	En pratique oui — le correcteur doit recréer ta base

À retenir : `sql/schema.sql` + `sql/reset-data.sql` = le minimum pour que le prof ou un client relance le projet chez lui.

Postman (docs/postman/) — pas présent, pas obligatoire

Question	Réponse
Présent dans ce projet ?	Non — pas de collection Postman
Obligatoire pour Spring Boot ?	Non
Obligatoire à l'examen ?	Souvent non si les tests automatiques suffisent

Alternatives à Postman : - Navigateur : `http://localhost:8081/api/articles` - Tests Maven :
`.\mvnw.cmd test` - `curl` en ligne de commande

Résumé

Élément	Obligatoire technique ?	Obligatoire pour le rendu ?
<code>sql/</code> (racine)	Indispensable pour exécuter	Oui
<code>docs/postman/</code>	Non (absent ici)	Non — optionnel pour la démo API

9. Installer et lancer le projet

Prérequis

- Java 21+
- MySQL (Laragon, port **3306**)

1. Créer la base

Dans HeidiSQL, exécuter dans l'ordre : 1. `sql/schema.sql` 2. `sql/reset-data.sql`

2. Lancer l'application

```
cd c:\laragon\www\spring-tumonakiese-henock
.\mvnw.cmd spring-boot:run
```

3. Tester dans le navigateur

Page	URL
Accueil	<code>http://localhost:8081</code>
Catégories	<code>http://localhost:8081/categories</code>
Auteurs	<code>http://localhost:8081/auteurs</code>
Articles	<code>http://localhost:8081/articles</code>
Commentaires	<code>http://localhost:8081/commentaires</code>
Recherche	<code>http://localhost:8081/auteurs?keyword=Henock</code>

4. Tester l'API

Dans le navigateur : `http://localhost:8081/api/auteurs`

5. Tests automatiques

```
.\mvnw.cmd test -DforkCount=0
```

10. Préparer une soutenance

1. **Le domaine** : à quoi servent catégories, auteurs, articles, commentaires
2. **Les 3 couches** : montrer Controller → Service → Repository
3. **Un JOIN** : ouvrir `AuteurRepoImpl.java` et expliquer la requête SQL
4. **simpleflatmapper** : pourquoi on l'utilise pour les JOIN
5. **DTO vs Modèle** : le DTO valide l'entrée, le modèle représente la table
6. **Démo live** : créer un article, publier un commentaire, chercher « Henock »
7. **L'API** : montrer `GET /api/auteurs` et le JSON retourné

11. Technologies utilisées

Technologie	Rôle
Java 21	Langage
Spring Boot 4.x	Framework
Spring JDBC (JdbcClient)	Accès MySQL
simpleflatmapper	Mapping JOIN → objets Java
Thymeleaf	Templates HTML
Bootstrap 5	Mise en page responsive
MySQL 8	Base de données
Maven	Build (<code>mvnw</code>)
JUnit 5	Tests d'intégration

12. Glossaire

Terme	Signification
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object — données échangées formulaire/API
JOIN	Requête SQL qui combine plusieurs tables
Endpoint	URL de l'API (<code>/api/auteurs</code>)
Profil Spring	Configuration par environnement (<code>dev</code> , <code>prod</code>)
Thymeleaf	Moteur de templates HTML côté serveur
Repository	Couche d'accès aux données

13. Liens utiles

- [Documentation Spring Boot](#)
- [Thymeleaf](#)
- [simpleflatmapper](#)
- Repo GitHub : <https://github.com/Xenoblade-lab/spring-TUMONAKIESE-HENOCK>

Document rédigé pour Tumonakiese Henock — Projet Blog Henock.